Streaming Fairness Monitor

Technical report | Reproducible study, Apache Flink, UCI Adult | November 2026

1. Problem and motivation

Fairness can change after a model is deployed, even when a batch evaluation looked acceptable. A practical monitoring service should compute fairness indicators repeatedly over recently arriving decisions, and raise alerts when disparities exceed a chosen reference. This project implements an educational monitoring pipeline for fictional loan approvals using the UCI Adult census-income dataset as a proxy task.

The project is **not** trained on actual financial loan records. Adult predicts income above USD 50,000. The binary income prediction is deliberately reinterpreted only for a controlled classroom scenario. A synthetic stream time and an injected phase shift create a known drift point against which alert speed can be measured.

2. Architecture

Dataset preparation: download Adult files, combine and clean, fixed seed 42, 70/30 split, one-hot encode categorical predictors and standardize numeric predictors, then fit logistic regression without the explicit sex feature. Sex is retained only for group measurement. This does not remove proxy effects.

Streaming path: held-out CSV to Python TCP replayer to PyFlink socket source to sliding processing-time windows to fairness metrics and alerts to JSONL output to Streamlit. Each emitted record includes a synthetic replay timestamp, true income label, predicted proxy decision, group and drift-phase marker.

Independent experiment: an offline event-count sliding-window evaluator estimates the reference gaps on pre-shift windows. It repeats measurement at different sample sizes and writes CSV, JSON and a plot. This controlled benchmark is not the same as wall-clock Flink monitoring.

3. Dataset and ethics

UCI Adult: 48,842 rows and 14 features (source inventory); CC BY 4.0; 1994 US Census-derived data. The code downloads official adult.data and adult.test files; raw data is not bundled. This data has historical and representation limitations and is inappropriate for operating a real loan approval system. No individual identification is attempted.

Metrics and streaming design

4. Fairness metrics

Demographic parity (DP) measures the group difference in rates of favorable predicted decisions: DP = P(pred = 1 | Male) - P(pred = 1 | Female). **Equal opportunity (EO)** measures the group difference in true-positive rates: EO = P(pred = 1 | income label = 1, Male) - P(pred = 1 | income label = 1, Female). Positive gaps favor Male, zero indicates equal rates, and negative gaps favor Female.

A DP estimate is suppressed when either group has fewer than 10 records. EO is suppressed when either group has fewer than 5 true-positive-eligible records. The code returns missing values instead of substituting zeros. Fairness gaps are descriptive and do not independently establish unfair treatment.

5. Windowing and alerts

The PyFlink program uses **SlidingProcessingTimeWindows** with a 20-second length and 5-second slide by default, parallelism 1, and a single aggregation over the current window. This is an intentionally small teaching workload. The source is TCP, so arriving rows are processed according to wall clock rather than original (synthetic) event time. Scheduling delays and stream rate can change window membership.

The live alert rule is abs(observed gap - configured reference gap) >= 0.12. For the Docker demo the reference gaps default to zero. That makes this a **target-based** alert, not a statistically calibrated change detector. The offline experiment instead uses empirical pre-shift gap averages as its reference. These must not be conflated.

6. Controlled experiment

The deterministic synthetic pilot contains 8,000 simulated records; each has a Male/Female group, a simulated true label, and a noisy prediction. At event 4,000 a biased mechanism flips 75% of otherwise positive Female predictions to negative. Each study uses an event-count sliding window with a slide of 100 rows, sizes 200, 500, 1,000 and 2,000, and an absolute deviation threshold of 0.13.

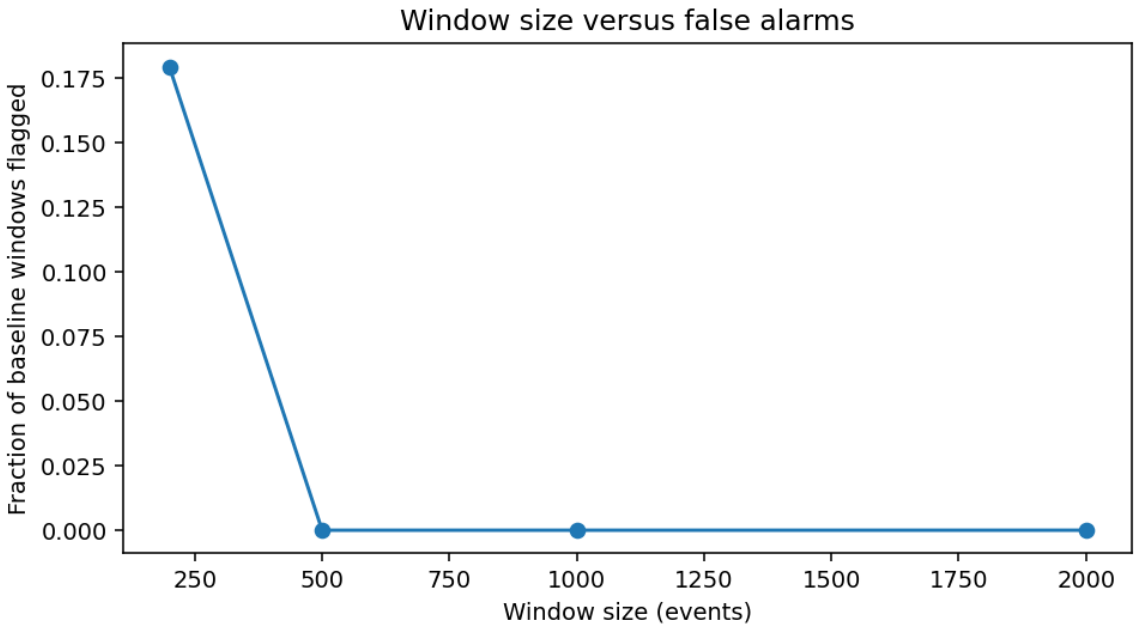
A false alarm means an alert in a window ending at or before the injection point; detection delay counts events from injection to the first subsequently flagged window. Windows spanning both baseline and drift are classified as post-drift, which makes immediate detection easier to score. Baseline windows are correlated because they overlap.

Reproduction: run `python -m src.study --dataset synthetic`. Run `python -m src.study --dataset adult --out results/study_adult.csv` after acquiring official data and creating held-out events.

Observed pilot results

7. Synthetic benchmark

Window (events)	False alarms / rate	Detection delay	Drift alert rate
200	7 / 17.9%	100 events	100.0%
500	0 / 0.0%	100 events	100.0%
1000	0 / 0.0%	200 events	97.5%
2000	0 / 0.0%	400 events	92.5%



Interpretation. The 200-event window generated 7 false alarms in 39 baseline windows (17.95%). Sizes 500, 1,000 and 2,000 generated zero baseline false alarms in this single seeded run. Detection took 100 events at sizes 200 and 500; 200 events at size 1,000 and 400 events at size 2,000. The larger windows reduce random fluctuation but dilute newly arriving biased outcomes.

Limitations of these numbers. This is one deterministic synthetic trial, not a confidence interval or broad generalization. All outputs are generated from synthetic records, not from the actual Adult dataset and not from measured Flink execution latency. Window size in this table is measured in event counts. Change the seed, anomaly severity, threshold, or source rate and results will change.

Reproducibility and conclusions

8. Reproducibility and validation

Run `python -m src.prepare_data` to download UCI Adult, fit the deterministic model, and create replay data. Run `docker compose up --build` to start Flink JobManager, TaskManager, TCP replayer, submitter, and Streamlit. Flink is exposed at port 8081 and Streamlit at port 8501. Run the Python tests with `python -m pytest -q`. Use the Jupyter notebook to examine CSV outputs. Installation and troubleshooting commands are included in the repository.

Validation status at package authoring: four Python unit tests passed and the deterministic synthetic pilot was executed. The UCI source download, Docker image build, live Flink execution and Streamlit UI were not run in the authoring environment; they require user-machine network and Docker access. Therefore, no live runtime screenshots or performance claims are supplied.

9. Engineering risks and improvements

The teaching pipeline uses a socket source and a side-effect file sink. It does not provide replayable data retention, exactly-once delivery or transactional output. A production-grade design would use Kafka, event-time watermarks, Flink checkpoints, a durable append log, deduplication, a metrics store, alert persistence and deployment monitoring. Confidence intervals, multiple testing correction and adequate subgroup sample sizes should precede operational fairness alarms.

An especially important calibration improvement is to monitor **deviations from a learned stable baseline** instead of setting a blanket zero target. Fairness is context-specific: demographic parity and equal opportunity can conflict when outcome prevalence differs. A human review process and domain-specific legal validation are essential for real high-impact decisions.

10. Conclusion

The artifact provides a runnable educational streaming architecture and a window-size benchmark. The synthetic results illustrate a stability-versus-detection-delay trade-off: longer windows suppressed nuisance alarms in this pilot, but their alert response came later. The report intentionally separates simulated evidence, offline event-count evaluation and unverified live Flink execution.

References

Becker, B. and Kohavi, R. (1996). *Adult*, UCI ML Repository. DOI: 10.24432/C5XW20. CC BY 4.0. Apache Flink documentation, DataStream windows and Docker deployment. Code, dataset card and detailed setup are included with this submission.